

SE 310 Software Design

Lab: MazeGame with the Factory Method & the Abstract Factory Patterns

Goal:

1. Practice Factory Method Pattern
2. Practice Abstract Factory Pattern

Requirements:

1. Adding the `RedMazeGame` and `BlueMazeGame` features using the Factory Method and Abstract Factory design patterns to the basic maze created for the homework 1 assignment. `RedMazeGame` will use pink rooms and red walls. `BlueMazeGame` will use green rooms, blue walls, and brown doors.
2. Compile the new program with the ability to show basic, red, or blue maze games, reading the layout from the provided `small.maze` and `large.maze` files using the Factory Method Pattern.
3. Modify and compile the maze game to show red, or blue maze games, reading the layout from the provided `small.maze` and `large.maze` files using the Abstract Factory Pattern.

Instructions:

Stage 1: Using the Factory Method Pattern

1. Copy your basic maze code from the homework to a new directory and create a new IntelliJ project. **Make sure you keep a version of the homework code separate from this lab.**
2. Modify the basic maze according to the UML Factory Method found below (see Figure 1)
 - a. Modify `SimpleMazeGame` into a `MazeGameCreator` class by adding factory methods and change the `main` method accordingly
 - b. Run
3. Add the necessary code for the `RedMazeGame` features
 - a. Add the new components, `RedWall`, `PinkRoom`
 - b. Add a new `RedMazeGameCreator` class
 - c. Run the program to create a red maze
4. Add the necessary code for the `BlueMazeGame` features
 - a. Add the new components, `BlueWall`, `GreenRoom`, `BrownDoor`
 - b. Add a new `BlueMazeGameCreator` class
 - c. Run the program to create a blue maze

5. The program should now allow you to create a basic, red, or blue maze depending on input parameters. You should prompt the user to choose between the options: basic, red, or blue. You should default to parsing the `large.maze` file. If you cannot parse the `large.maze` file you may use the `small.maze` file. If you cannot parse either file, please see the TA or professor.
6. **Save this code separately**, it will need to be submitted to show you properly completed the Factory Method design pattern.

Stage 2: Using the Abstract Factory Pattern

1. Create a new working directory and a new IntelliJ project. Copy the code you created for the Factory Method pattern to this new directory.
2. Modify the code you copied to follow the UML Abstract Factory found below (see Figure
 - a. `MazeGameCreator` or `SimpleMazeGame` should become a `MazeFactory`, an abstract class
3. Add the necessary code for the `RedMazeGame` features
 - a. Add a `RedMazeFactory`
 - b. Red maze components, if not copied over
4. Add the necessary code for the `BlueMazeGame` features
 - a. Add a `BlueMazeFactory`
 - b. Blue maze components, if not copied over
5. Create a new `Driver`
 - a. The driver will have your `main` function
 - b. The driver will have a `loadMaze (String path, MazeFactory factory)` function. The `loadMaze` function takes the path to the file to parse and a factory, either Red or Blue. The `loadMaze` function from the homework 1 lives in the `MazeFactory` that `Red/BlueMazeFactory` inherits from.
6. The program should now allow you to create a red or blue maze depending on input parameters. You should prompt the user to choose between the options: red or blue. You should default to parsing the `large.maze` file. If you cannot parse the `large.maze` file you may use the `small.maze` file.
 - a. Please note that for this part of the lab we are no longer creating a basic maze, we are only creating red or blue mazes.

Grading Guidelines

1. Applying Factory Method pattern
 - a. (50%) The code conforms to the UML class diagram
2. Applying Abstract Factory Method pattern
 - a. (50%) The code conforms to the UML class diagram

Factory Method Pattern UML:

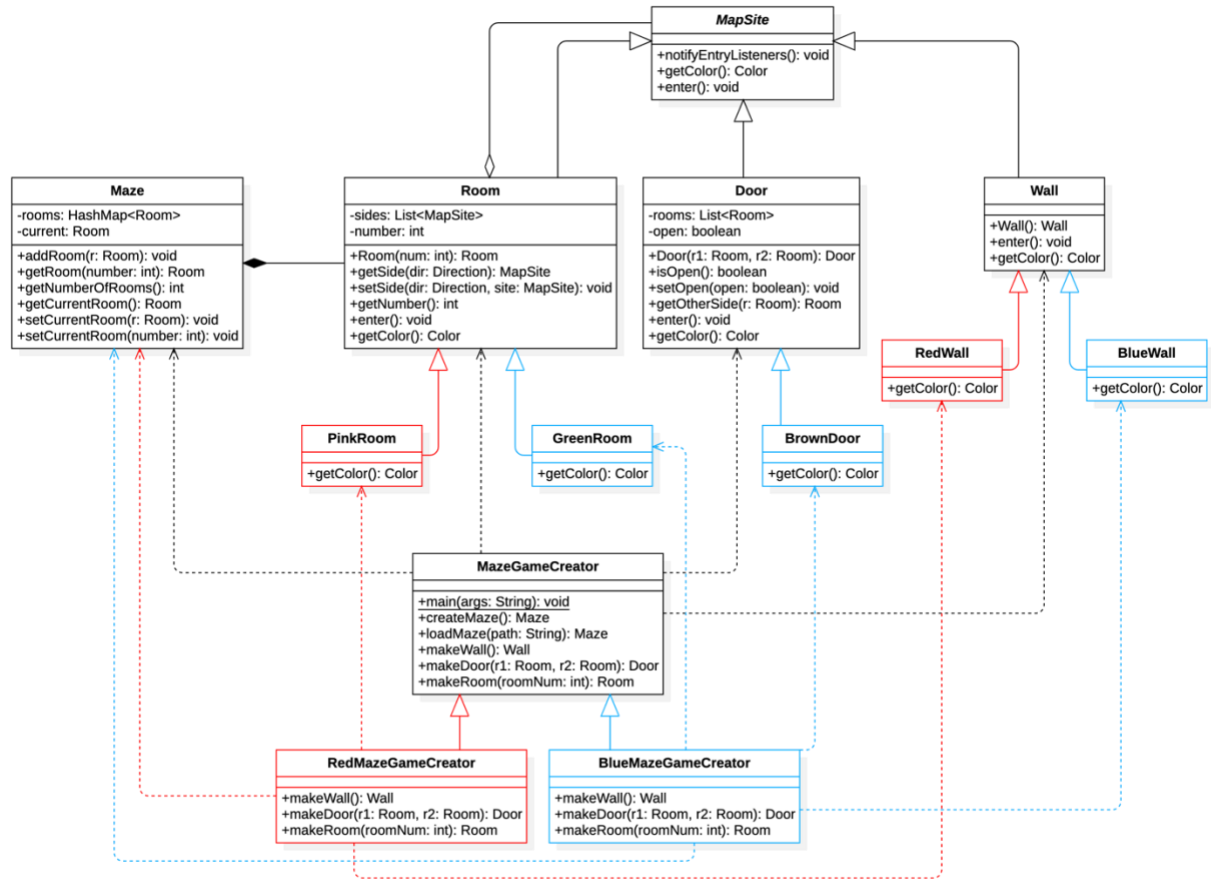


Figure 1

Abstract Factory Pattern UML:

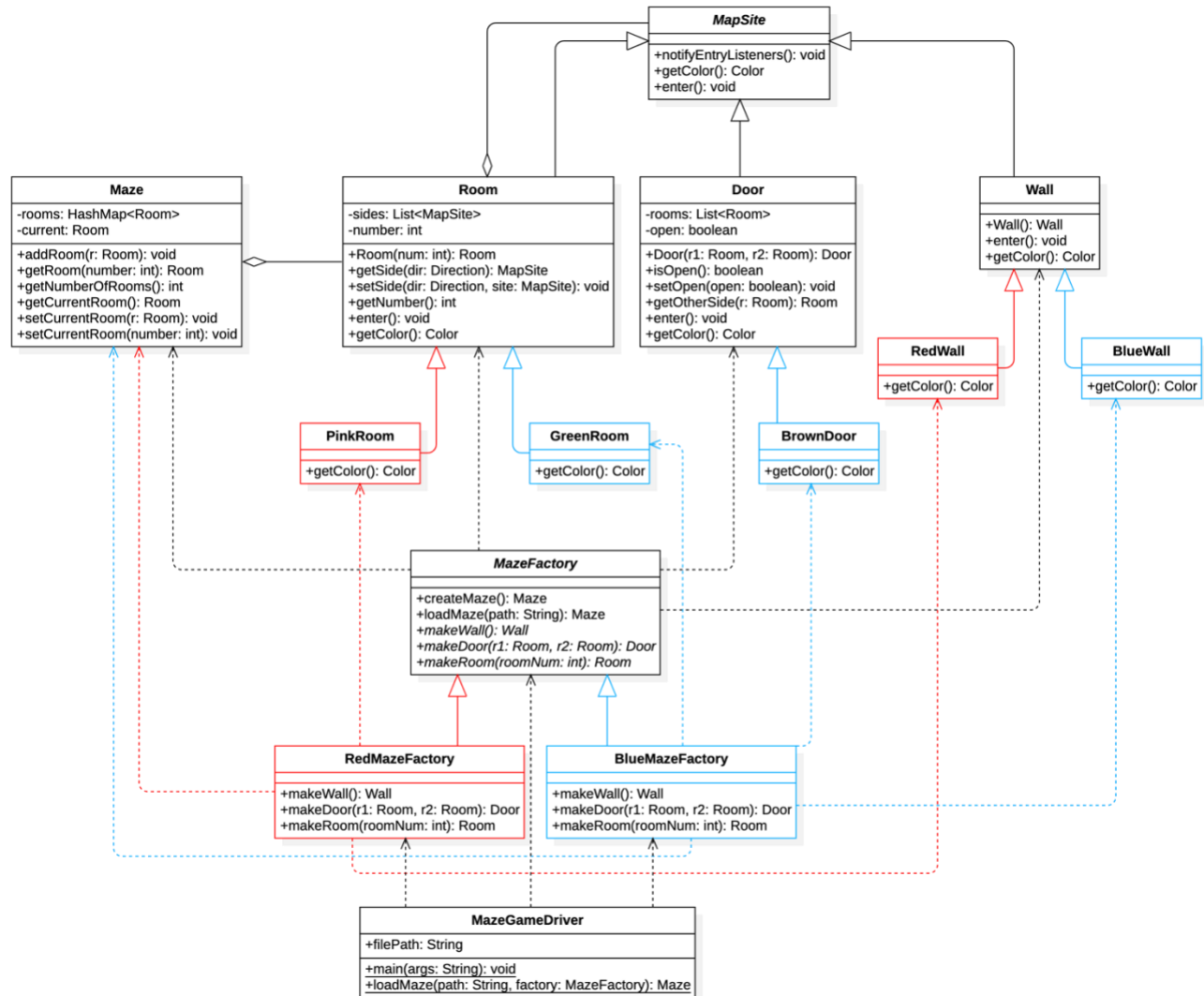


Figure 2